Evaluating Password Strength Meters Against Simulated Guessing Attackers: Human, Random and AI-Generated Passwords

AINOUCHE Rayan (Nº 145882), JOVANOVIC Marko (Nº 145885),
GERMAIN Valentine (Nº 146069), ADYEL Ilyes (Nº 145869)
Iscte - Instituto Universitário de Lisboa
Network and Information Systems Security - Prof. José André Moura

***Abstract* - Password strength meters (PSMs) decide, at password-creation time, whether a password is good enough. A meter that rates a guessable password "strong" gives users false confidence. Earlier studies compare meters with simulated attackers, but, to our knowledge, none reports how often meters accept guessable passwords for human, random and large-language-model (LLM) generated passwords together, and none includes the Linux password-quality tools `libpwquality` and `passwdqc`. This paper is the theoretical part of our project. We review the state of the art and specify a protocol that measures the *dangerous false-positive rate*, the share of guessable passwords that a meter rates strong, for five meters (zxcvbn, composition rules, `libpwquality`, `passwdqc`, Have I Been Pwned). Ground truth is the minimum guess number over simulated attackers (dictionary, rule-based, LLM-aware) up to 10^{10} guesses. Pilot measurements show that the protocol is feasible on a 2-core machine: a 10^{10}-guess rule-based stream takes about 41 minutes, and a first end-to-end run with a dictionary attacker already separates the meters. Results are left to the practical part.**

***Index Terms* - password strength meter, guessability, zxcvbn, Have I Been Pwned, large language models**

I. Introduction

Passwords remain the default authentication mechanism, and users choose them under the guidance of password strength meters: composition-rule checkers, strength bars, or blocklist checks. A meter is only useful if its verdict matches real strength; in the words of Golla and Dürmuth, "a strength meter with low accuracy may do more harm than good" [1]. The harm is asymmetric. Rating a weak password strong gives false confidence, whereas rating a strong password weak mainly annoys the user [2].

Two developments motivate this work. First, NIST SP 800-63B-4 (July 2025) requires verifiers to compare new passwords with a blocklist of commonly used, expected or compromised passwords, and forbids other composition rules [3], while Golla and Dürmuth found composition-based (LUDS) meters, with poor accuracy, in several password managers and operating systems [1]. Second, users and coding assistants now ask LLMs to generate passwords. Irregular reports that 50 runs of one model produced only 30 distinct 16-character passwords, one of them 18 times, and estimates about 27 bits of entropy for such a password against about 98 bits for a random one; it also reports that KeePass and zxcvbn rated the password as very strong (about 100 bits, "centuries") [4]. This is a small, single-vendor case study, not a measurement against attackers.

Problem. How often do common meters accept passwords that a realistic attacker would guess, and does the answer depend on who or what created the password? We answer with guessability-based evaluation, now the standard method [1], [2], [5]: simulate attackers, compute a guess number for each test password, and compare it with the meter's verdict.

Research questions.

- **RQ1.** For each meter and each population (human, random, LLM), what share of the passwords guessed within a budget θ is rated strong (dangerous false positives)?
- **RQ2.** Does this share differ between populations, in particular for LLM passwords attacked by an LLM-aware attacker?
- **RQ3.** What does each meter cost in usability (strong passwords rejected), and how do both errors change with θ?

Contributions. (C1) A comparison of eight related works along the dimensions that matter for our question (Sec. II). (C2) A protocol with explicit definitions, equations and design choices justified by the literature (Sec. IV). (C3) Pilot measurements of throughput, redundancy and an end-to-end smoke test showing that the protocol is feasible (Sec. V). (C4) A nine-week plan, a repository structure and a list of threats to validity.

II. Background and Related Work

A. Guess numbers and ground truth

The *guess number* of a password p for an attacker A is the position at which the attacker's ordered candidate list first contains p (Eq. 3). Ur et al. compared guessing tools and found that running a single algorithm often gives a very poor estimate of strength; they therefore propose the minimum over several configured attacks (*min_auto*), which the CMU Password Guessability Service provides [1], [6]. Wheeler takes the minimum over four attacks (PCFG, Markov, hashcat, John the Ripper) as ground truth [5]; Wang et al. use the union of three attacks [2]. For probabilistic models, guess numbers beyond what can be enumerated can be estimated by

Monte Carlo sampling [7], which Wang et al. use to convert probabilities into guess numbers [2].

Training data matter. Wang et al. require that training and test sets never come from the same service [2]; Wheeler notes that his same-distribution split makes the simulated attacker aggressive [5]. Xu et al. (arXiv preprint) show that data-driven meters over-learn their training passwords, which then receive unusually high probabilities and can be recovered by membership inference [8].

B. Meter families and standards

Composition-rule (LUDS) meters count lower-case and upper-case letters, digits and symbols. Golla and Dürmuth measured them as inaccurate: the weighted Spearman correlation of Comprehensive8 with the reference is negative on RockYou and LinkedIn (online: -0.652 and -0.589; offline: -0.476 and -0.616) [1]. *Pattern-based estimators* such as zxcvbn match dictionary words, keyboard patterns, sequences, repeats and dates and estimate a guess count [5]; Wang et al. map such counts to zxcvbn-style scores with thresholds $10^3, 10^6, 10^8, 10^{10}$ [2]. *Blocklists* are mandatory in NIST SP 800-63B-4, which compares the *entire* password with a list of breached, dictionary and context-specific words and asks verifiers to explain a rejection [3]. Have I Been Pwned (HIBP) exposes such a list through a k-anonymity range API: the client sends the first five hexadecimal characters of the SHA-1 hash and receives the matching suffixes with their counts [9]. Golla and Dürmuth rank it among the most accurate meters (weighted Spearman 0.739-0.997) but note that this is probably due to the tested leaks being part of the list [1]. `libpwquality` and `passwdqc` are the password-quality libraries behind Linux PAM modules [10], [11]; they are not among the meters evaluated by Golla and Dürmuth or by Wang et al.

C. Measuring accuracy

Wheeler measures, in orders of magnitude, the gap between a meter's estimate and the ground-truth guess number on 15 k RockYou passwords. Below 10^6 guesses, zxcvbn's mean absolute error is 0.58 orders of magnitude (version with 100 k-token dictionaries), against 1.81 for a NIST-entropy estimator and 1.43 for KeePass; above 10^6 it grows to 2.20. Wheeler therefore presents the tool as accurate and conservative at low magnitudes, suited to online attacks [5].

Golla and Dürmuth test 19 similarity measures against five requirements and select the weighted Spearman correlation, with password frequency as weight; they compare 45 meters (81 variants) on RockYou, LinkedIn and 000Webhost, using sample frequency (top 10^4 pool) as online reference and PGS min_auto as offline reference [1]. They acknowledge that the measure does not distinguish under- from over-estimation and that quantized meters must be interpreted with care [1].

Wang et al. propose four metrics: weighted Spearman (online, threshold 10^4 guesses), KL divergence, Precision and PrecisionSecurity (offline, above 10^9 guesses), over 12 meters and 7 English and 7 Chinese leaks, with training and test sets from different services [2]. PrecisionSecurity weights weak→strong errors with $\beta = 0.8$, but, as printed, its β-weighted term is computed on the lowest-score set L, whereas weak→strong errors are the cracked passwords in the highest-score set H. We therefore measure the weak→strong error directly (Eq. 6).

Tan et al. run two MTurk experiments (4,317 and 4,005 participants completed password creation) comparing blocklists, minimum-strength requirements and composition rules, with guessability from PGS min_auto at 10^6 (online) and 10^{14} (offline) guesses; they recommend blocklists or minimum-strength requirements and advise against character-class requirements [12].

D. Which budgets matter

Florêncio et al. argue that guessing resistance matters only below an online limit $T_0 \approx 10^6$ and above an offline limit $T_1 \approx 10^{14}$; between them lies a "don't care" region, and they favour blacklists, throttling and hash iteration over composition policies [13]. Golla and Dürmuth cite 100-1000 online guesses per 30 days and 10^9-10^{12} offline guesses per day on one GPU [1]. Our budget $B = 10^{10}$ lies inside the "don't care" region; Sec. IV explains why we still use it.

E. LLM-generated passwords

Beyond the Irregular report [4] (a company blog post, not peer-reviewed), Atzori et al. (arXiv preprint) study LLMs that generate and evaluate passwords from social-profile data for 100 users; they rely on their own composite metric (CPS) and report that 22 of 25,000 strong generated passwords were found by a targeted guesser [14]. Neither work simulates attackers against standard meters.

F. Comparison and gap

Table I summarizes the works. None combines (i) Linux password-quality tools, (ii) a threshold-based, one-sided error measure for accept/reject verdicts, and (iii) human, random and LLM-generated passwords with an attacker that knows the LLM output distribution. Golla and Dürmuth note that their selected measure does not distinguish under- from over-estimation and that quantized outputs need careful interpretation [1]; a one-sided error for binary verdicts is what we add.

III. Problem Statement and Hypotheses

We evaluate the verdict $V_M(p) \in \{\text{strong}, \text{weak}\}$ of each meter M on three populations: human passwords (H), random passwords (R, control) and LLM-generated passwords (L). Targeted attacks, user-behaviour effects and meter feedback text are out of scope.

Our hypotheses are informed by the literature and by the pilot of Sec. V; they are to be tested, not assumed.

- H1 No meter dominates on both errors. Strict meters (composition, `passwdqc`) will show a low false-positive rate but reject most strong passwords, so FPR must be read together with FNR (the pilot already shows this).
- H2 The FPR of zxcvbn rises from the online budget to B, because the tool was validated only in the online

TABLE I
COMPARISON OF RELATED WORK. "BUDGET" IS THE NUMBER OF GUESSES OF THE GROUND-TRUTH ATTACKER.

Work	Meters evaluated	Ground truth (budget)	Passwords	Main metric	LLM
Wheeler 2016 [5]	zxcvbn, NIST entropy, KeePass	min of PCFG, Markov, hashcat, JtR (caps 10^{10}-10^{14})	15 k RockYou (+ Yahoo)	$\lvert\Delta\rvert$, Δ^+ in orders of magnitude	no
Golla 2018 [1]	45 meters (81 variants)	online: frequency, top 10^4; offline: PGS min_auto	RockYou, LinkedIn, 000Webhost	weighted Spearman	no
Wang 2023 [2]	12 meters	online 10^4; offline JtR+rules, Markov, brute force (10^9-10^{12}), union	7 English + 7 Chinese leaks; train $\neq$ test service	WSpearman, KL, Precision, PrecisionSecurity	no
Tan 2020 [12]	minimum-strength NN, blocklists, composition (user study)	PGS min_auto; 10^6 online, 10^{14} offline	passwords created by MTurk participants	% guessed, usability	no
Florêncio 2016 [13]	conceptual	$T_0 \approx 10^6$, $T_1 \approx 10^{14}$	none (cites earlier studies)	analytical	no
Xu 2025 [8]	11 meters (risks of the meter itself)	membership inference, meter-aware guessing	six leaks, incl. RockYou	MIA F1, accounts compromised	no
Irregular 2026 [4]	KeePass, zxcvbn (informal)	entropy estimates (character statistics, log-probabilities)	50 runs per LLM	entropy in bits	yes
Atzori 2025 [14]	zxcvbn, CKL_PSM, semantic PCFG, own tool	targeted guesser (PassBERT); own CPS metric	100 users' profiles	CPS, accuracy, F1	yes
This work	zxcvbn, composition, `libpwquality`, `passwdqc`, HIBP	min over dictionary, rules, LLM-aware ($B = 10^{10}$)	phpBB, random, LLM	dangerous FPR, FNR	yes

range [5]. As an order of magnitude, our reading of the cell counts of Fig. 2(b) in [1] (zxcvbn score against the PGS min_auto guess number, sample of 10,000 passwords) gives 51 of 1,108 passwords with score 4 and a guess number below 10^{10} (4.6 %).

H3 Most LLM samples are rated strong by pattern- and composition-based meters [4] while the LLM-aware attacker guesses a large share of them; HIBP rates nearly all of them strong, since fresh LLM output is unlikely to be in breach corpora.

IV. METHODOLOGY

Fig. 1 summarizes the pipeline and Table V lists each design decision with its source.

A. Meters under test

Table II fixes one binary verdict per meter. For graded meters (zxcvbn guess number, `pwscore` score) we also sweep the threshold and report FPR against FNR curves.

zxcvbn estimates the guess count of a password as the minimum, over non-overlapping covers S of the password by matched patterns, of [5]

$$\hat{g}(p) = \min_S \Big[\, |S|! \prod_{m \in S} g(m) + D^{|S|-1} \Big], \quad D = 10^4, \qquad (1)$$

where $g(m)$ is the guess estimate of match m (for unmatched characters, 10^ℓ for ℓ brute-force characters). The 0-4 score is the number of thresholds exceeded (the thresholds come from the zxcvbn source code, not from [5]),

$$s(\hat{g}) = \sum_{t \in \{10^3, 10^6, 10^8, 10^{10}\}} \mathbf{1}[\hat{g} \geq t + 5]\,. \qquad (2)$$

TABLE II
METERS AND THE BINARY VERDICT "STRONG".

Meter	Implementation	Strong iff
zxcvbn	Python package 4.5.0, reference year pinned to 2026 (Sec. V)	score ≥ 3 ($\hat{g} \geq 10^8$)
Composition	own function	length ≥ 8 and ≥ 3 of 4 classes
`pwscore`	`libpwquality` tools [10], default configuration	password accepted (exit code 0)
`pwqcheck`	`passwdqc` [11], default configuration	password accepted (exit code 0)
HIBP	range API, 5-hex prefix, `Add-Padding: true` [9]	count $= 0$

We checked Eq. (2) against the Python package (version 4.5.0) on 2,000 random passwords: no mismatch.

B. Password populations

Table III lists the populations. Attackers are trained on RockYou and tested on a different service (phpBB), as required by [2]; Wang et al. also train on RockYou. Passwords are ASCII, 8-64 characters: 8 is the NIST minimum for passwords used with multi-factor authentication [3], whereas the 15-character single-factor minimum would keep only 0.18 % of the phpBB accounts (463 of 255,420). Evaluation is at the *account* level: a password used by c accounts counts c times, so that each observation is an independent draw from the user population. Random passwords are drawn uniformly from the 94 printable non-space ASCII symbols with Python's `secrets`; their entropy is $L \log_2 94$ bits (52.4, 78.7, 104.9 bits for $L = 8, 12, 16$). LLM passwords come from three open-

TABLE III
PASSWORD POPULATIONS. COUNTS FOR ROCKYOU AND PHPBB ARE FROM THE SECLISTS COPIES [16] BEFORE/AFTER OUR FILTER.

Group	Source	Size / role
RockYou	leak, all lengths	14,344,391 unique, 32,603,388 accounts; attacker training only
H	phpBB leak, ASCII, 8-64 chars	95,837 unique, 116,289 accounts; test
R	`secrets`, 94 symbols, $L \in \{8, 12, 16\}$	$3 \times 10{,}000$; control
L-train	LLM, per (model, prompt)	1,000; builds attacker A4
L-test	LLM, per (model, prompt)	1,000; test

weight instruction-tuned models of at most 4 billion parameters, run locally on CPU with Ollama [15] so that generation costs nothing (Sec. V-C; the list is fixed after the week-1 benchmark), with two prompts: a neutral one ("Please generate a password", as in [4]) and one that asks for a strong 16-character password; each generation is an independent session at temperature 0.7, the value used in [4].

C. Ground-truth attackers

For an attacker A producing a candidate stream $(c_1, c_2, \dots)$ truncated at budget B,

$$G_A(p) = \begin{cases} \min\{k \le B : c_k = p\} & \text{if } p \in \{c_1, \dots, c_B\}, \\ \infty & \text{otherwise.} \end{cases} \quad (3)$$

The ground-truth guess number and the label at budget $\theta \le B$ follow the min_auto principle [6]:

$$G(p) = \min_{A \in \mathcal{A}} G_A(p), \qquad y_\theta(p) = \mathbf{1}[G(p) \le \theta]\,. \quad (4)$$

We compute $G(p)$ once with $B = 10^{10}$ and evaluate every $\theta \in \{10^4, 10^6, 10^8, 10^9, 10^{10}\}$ afterwards, at no extra cost. The headline budgets are $\theta_{\text{on}} = 10^6$ (online limit of [12], [13]) and $\theta_{\text{off}} = B = 10^{10}$; 10^4 is Wang's online threshold [2], 10^8 and 10^{10} are the claims of zxcvbn scores 3 and 4, and 10^9 is Wang's offline floor. At the rates cited in [1] (10^9-10^{12} guesses per day and GPU), B costs between 15 minutes and 10 GPU-days. We do not reach the 10^{14} offline limit of [12], [13]: enumerating it would take about 285 days at our measured rate. Our guess numbers are therefore upper bounds on strength, and the measured FPR are expected to be lower bounds on the true danger.

Ur et al. show that a single default configuration underestimates guessability [6], so A2 uses two configurations and the minimum is kept. A4 knows the model and prompt but not the test samples; for ties in frequency, G_{A4} is the end of the tie group, which is conservative. Hashcat enumerates words in the outer loop and rules in the inner loop, and its streams contain duplicates; G_A is the raw stream position and therefore upper-bounds the deduplicated rank (Sec. V).

TABLE IV
ATTACKERS. COST: MEASURED RATE OF SEC. V (A2A EXTRAPOLATED).

ID	Candidates	Budget	Cost
A1	RockYou list by decreasing frequency	1.4×10^7	seconds
A2a	hashcat [17] `best64` (77 rules) on the full list	1.10×10^9	≈ 31 min
A2b	hashcat `dive` (99,086 rules) on the top 10^5 words	9.91×10^9	≈ 41 min
A4	LLM-aware: outputs of the same (model, prompt) in L-train, by decreasing frequency	$\le 10^3$	seconds

Simulated brute force is omitted on purpose. With 94 symbols, the number of passwords of length at most L is

$$N_{BF}(L) = \sum_{k=1}^{L} 94^k. \quad (5)$$

Since $N_{BF}(5) = 7.42\times10^9 < B < N_{BF}(6) = 6.97\times10^{11}$, brute force reaches only lengths ≤ 5 within B, and all our test passwords have at least 8 characters. The expected guess number of a random 8-character password is $94^8/2 \approx 3.0\times10^{15} \gg B$: random passwords are unguessed by construction, and serve as a control for the meters' strong rate.

D. Metrics

Let a "positive" be a strong verdict. For a population P of accounts and a budget θ:

$$\mathrm{FPR}_\theta(M; P) = \frac{\#\{p \in P : V_M(p) = \text{strong} \wedge y_\theta(p) = 1\}}{\#\{p \in P : y_\theta(p) = 1\}}, \quad (6)$$

$$\mathrm{FNR}_\theta(M; P) = \frac{\#\{p \in P : V_M(p) = \text{weak} \wedge y_\theta(p) = 0\}}{\#\{p \in P : y_\theta(p) = 0\}}. \quad (7)$$

Eq. (6) is the *dangerous false-positive rate*, our primary metric: it does not depend on how many guessable passwords the population contains, unlike the share of guessed passwords among those rated strong. FNR is the usability cost. Because ground truth under-estimates guessability, FNR is expected to be inflated and FPR to be a lower bound. For R, FPR is undefined (no password is guessed), so we report the strong rate and the counts.

Each proportion $\hat{p} = k/n$ comes with a Wilson 95 % interval ($z = 1.96$),

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \quad (8)$$

where n is the number of guessed accounts for FPR. With $n = 1{,}000$ the half-width is at most 3.1 points; with $n = 300$, at most 5.6 points. To compare two meters on the same guessed passwords, an exact McNemar test uses the b passwords rated strong by M_1 only and the c rated strong by M_2 only; under H_0, $b \sim \mathrm{Bin}(b+c, \frac{1}{2})$. For graded outputs we add the weighted

TABLE V
DESIGN DECISIONS AND THEIR SOURCES.

Decision	Source
Ground truth = minimum over several attackers	Ur 2015 [6]; Wheeler [5]; Wang [2]
Attackers trained on a service other than the test service	Wang [2]; Wheeler's caveat [5]
Observations weighted by account frequency	Golla [1]
Online budgets 10^4, 10^6	Wang [2]; Florêncio [13]; Tan [12]; Golla [1]
Offline-style budget $B = 10^{10}$	zxcvbn score 4; GPU rates in [1]; our measured rate
One-sided error (strong rate among guessed)	Wang's weak→strong argument [2]; limitation noted in [1]
HIBP verdict on the entire password	NIST blocklist rule [3]
Minimum length 8; blocklist verdict without composition rules	NIST [3]
Meters binned to accept/reject, plus threshold sweeps	Golla on quantization [1]; Wang on binning [2]

TABLE VI
PILOT MEASUREMENTS ON THE REFERENCE MACHINE.

Measurement	Result
zxcvbn (Python), passwords per second	2,049
`pwscore` / `pwqcheck`, calls per second	539 / 640
hashcat `dive` stream into a Python consumer, candidates per second (198,172,317 candidates in 48.8 s)	4.06×10^6
hashcat `best64` on 200 k words, candidates per second	5.9×10^5
Unique / total candidates for 40 random top-20 k RockYou words: `best64`, `T0XlCv2`, `d3ad0ne`, `dive`	0.945, 0.734, 0.589, 0.492
Time for 10^9, 10^{10}, 10^{11}, 10^{14} candidates at 4.06×10^6/s	4 min, 41 min, 6.8 h, 285 d

Spearman correlation [1], the weighted Pearson correlation of ranks:

$$\rho_w = \frac{\sum_i w_i (R_i - \bar{R}_w)(S_i - \bar{S}_w)}{\sqrt{\sum_i w_i (R_i - \bar{R}_w)^2 \, \sum_i w_i (S_i - \bar{S}_w)^2}}, \qquad (9)$$

where R_i and S_i are the average ranks, among accounts, of $G(p_i)$ and of the meter's strength output (a password with frequency w_i counts w_i times; ties get average ranks). With these ranks Eq. (9) equals Spearman's coefficient on the account-level data.

V. FEASIBILITY AND ACTION PLAN

A. Pilot measurements

We measured the components on a reference machine (2 vCPU, 7 GB RAM, not the team VMs); Table VI gives the results. A 10^{10}-guess stream is feasible in under an hour, and the meters process 10^5 passwords in minutes. Two more checks were possible. A Pwned Passwords range query for the prefix `5BAA6`, made through a web-fetch tool because the shell could not reach the API, returned the list of SHA-1 suffixes with counts without any key, including the suffix of "password" (52,372,427 occurrences on 30 Oct. 2026) [9]. And the zxcvbn Python port matches the JavaScript reference once its reference year is aligned (see below). The LLM generation speed could not be measured here (model registries were unreachable); Sec. V-C bounds it and week 1 measures it.

The duplicate ratio shows that raw stream positions overstate the deduplicated rank by up to a factor ≈ 2 (0.3 orders of magnitude) for `dive`; we keep raw positions (what a naive attacker pays) and run a deduplicated variant on a sample as a robustness check.

Sanity checks. We recomputed the formulas independently: the Wilson interval (Eq. 8) against `statsmodels`; Eq. (9) against SciPy's Spearman coefficient on account-level data (difference below 10^{-16}), which also shows that the classical $1-6\sum d^2/(n(n^2-1))$ formula is invalid with ties; the zxcvbn thresholds of Eq. (2); and the Monte Carlo rank estimator of [7] on a toy Markov model of 97,655 strings with exact ranks (200 repetitions of 10^4 samples: bias below 0.3 % and relative standard deviation of about 2-3 % up to rank 10^4, but 138 % at rank 90,000 of 97,655). The estimator is therefore kept as future work, not as part of the ground truth.

zxcvbn parity. We compared the Python port with the JavaScript reference (npm version 4.4.2) on 3,012 passwords: 3,000 drawn at random from phpBB with a fixed seed and 12 hand-picked strings. As shipped, the two disagree on the guess estimate of 280 passwords (9.3 %, up to 47 % relative difference) and on the score of 3 (0.1 %). The cause is the reference year used for date patterns: the Python port fixes `REFERENCE_YEAR` at 2017, whereas the JavaScript code uses the current year. With the year set to 2026 in Python, the outputs agree on all 3,012 passwords (relative difference below 10^{-15}). Our wrapper therefore pins the year, which also makes the results independent of the date of the run.

B. End-to-end smoke test

To check the whole pipeline, we ran the four locally available meters on H against attacker A1 alone (Table VII). A1 guesses 12,099 of 116,289 accounts (10.4 %) within 10^4 guesses and 34,803 (29.9 %) within 10^6; its whole list (1.4×10^7 guesses) contains 44,846 accounts (38.6 %). This is not a result of the study: A2, A4 and the full set of populations are missing, so FPR is expected to be a lower bound and FNR to be inflated. It shows two things. The meters differ mainly in strictness (strong rate from 4.5 % to 63.0 %), so FPR alone would reward a meter that rejects everything, which justifies reporting FNR and curves. And a dictionary alone is not enough: variants such as capitalized or digit-suffixed words are the ones that can pass composition rules, which is why A2 is part of the protocol [6].

C. Zero-cost resources

The project must cost nothing: no paid API, no new subscription, no rented machine. Table VIII lists each resource. The only paid assets the team already holds are a Claude Pro and a Gemini student subscription, and the pipeline does not

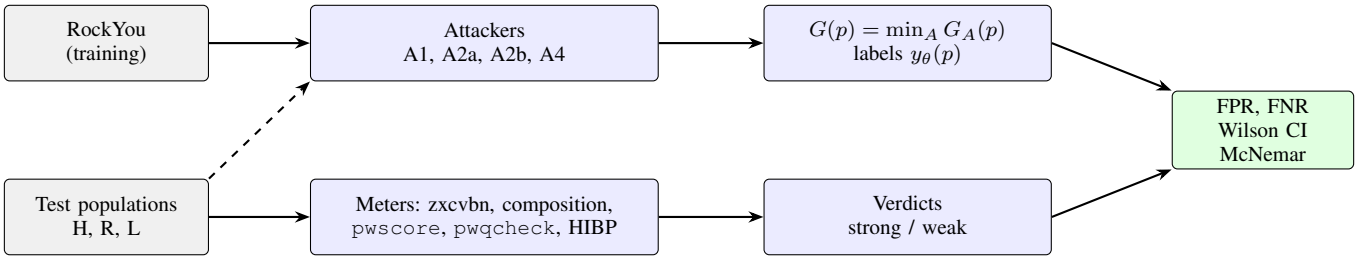


Fig. 1. Evaluation pipeline. Attackers are trained on RockYou (and, for A4, on L-train) and guess the test populations (dashed); meters score the same passwords; both outputs meet in the error metrics.

TABLE VII
SMOKE TEST: ATTACKER A1 ONLY, H (116,289 ACCOUNTS, PHPBB, ≥ 8 CHARS). WILSON HALF-WIDTHS ARE AT MOST 0.5 POINTS. STRONG RATE IS OVER ALL ACCOUNTS.

Meter	Strong rate	FPR 10^4	FPR 10^6	FNR 10^6
zxcvbn (score ≥ 3)	15.3 %	0.1 %	2.5 %	79.3 %
Composition	7.6 %	0.1 %	0.5 %	89.4 %
`pwscore` (accepted)	63.0 %	7.1 %	25.6 %	21.0 %
`pwqcheck` (accepted)	4.5 %	0.0 %	0.0 %	93.6 %

depend on either. Claude Pro does not include API access, which is billed separately [18], so Claude models can enter only through a small manual sample.

LLM generation costs time, not money. For one sequence, each decoded token reads the whole quantised model from memory, so the speed is at most the memory bandwidth divided by the model size. Dual-channel DDR4-3200 gives $3200 \times 10^6 \times 8 \times 2 = 51.2$ GB/s; the default Llama 3.2 3B build is 2.0 GB [15], hence at most 25.6 tokens/s, whereas a 7B-class model (about 4 GB) is capped near 13 tokens/s. We therefore restrict the models to 4 billion parameters. The population needs 3 models $\times$ 2 prompts $\times$ 2 sets $\times$ 1,000 = 12,000 generations. Charging 50 prompt tokens and a 64-token answer cap to every generation at the decoding speed, one generation takes $114/v$ s: the 12,000 generations need 19 machine-hours at $v = 20$ tokens/s and 38 at $v = 10$, i.e. 4.8 or 9.5 hours per machine on four team laptops, run overnight. The week-1 gate measures v on each laptop; below 10 tokens/s, we cut N to 500 or use two models.

Optional external-validity samples. (i) The Gemini API has a free tier without credit card for Flash models [19]; a request is billed only if its project is linked to an active billing account [20], so we would never link one. Free-tier content may be used to improve Google products and read by reviewers [20], which is harmless here since we send a generation prompt and no real password; its limits are shown in Google AI Studio and are not guaranteed, so it is used only if the quota suffices. (ii) Fifty generations per assistant (the number of runs in [4]) typed by hand in new conversations of the Claude Pro and Gemini chat applications, never scripted. These samples feed only the duplicate-rate and verdict comparison, not attacker A4.

TABLE VIII
RESOURCES AND THEIR COST.

Resource	Role	Cost
SecLists leaks [16]	H, attacker training	free, public
hashcat [17]	attackers A2a, A2b	free
zxcvbn, `libpwquality`, `passwdqc`	meters	free (installed in the pilot)
HIBP range API [9]	meter	free, no key
Ollama, 3 open-weight models ≤ 4 B	L population	free, local CPU
Gemini API free tier	optional sample	free without billing account
Claude Pro, Gemini chat	optional manual samples	already owned, no extra
Team laptops / VMs	all computation	no cloud rental
GitHub, local LaTeX	repository, report	free

D. Nine-week plan

Table IX gives the plan. Compute is not the bottleneck: both attack streams take about 1.2 hours, and the meters about 4 minutes on H (smoke test: about 40 s for zxcvbn and about 160 s for the two Linux tools with two processes). The main remaining risk is LLM generation time, which has a week-1 gate; the HIBP access and the zxcvbn parity were tested in the pilot.

E. Repository and ethics

The repository, to be created at the start of the practical part, will contain `src/meters`, `src/attackers`, `src/llm`, `src/metrics`, `tests/`, `results/` (aggregated counts only), `paper/` and a `Makefile`; library versions are pinned. We use only public leaks distributed in SecLists [16], keep raw leak files on the VMs and out of the repository, do not try to identify individuals, send only five-hex-character SHA-1 prefixes to HIBP [9], and publish no credential. No paid service is used: no billed API key and no cloud rental (Sec. V-C).

VI. THREATS TO VALIDITY

- **Ground truth.** Our attackers are weaker than the best ones and capped at 10^{10}; FPR are expected to be lower bounds. Raw stream positions over-state guess numbers by up to $\approx 2\times$.

TABLE IX
NINE-WEEK PLAN.

Wk	Work and deliverable
1	Machines, repository skeleton; **gates**: LLM throughput benchmark on each laptop ($\geq$ 10 tokens/s, else $N = 500$ or two models), HIBP test with 100 passwords from a team machine; zxcvbn parity kept as a unit test
2	Data filters, meter wrappers with `pytest`; verdict table for H
3	Attackers A1, A2a, A2b: streaming consumer, deduplicated sample; $G(p)$ for H
4	LLM generation (batched), A4, random control R
5	Full run: all meters $\times$ populations; HIBP lookups with prefix cache; master table
6	Metrics: FPR, FNR, Wilson, McNemar, threshold sweeps; figures
7	Robustness: HIBP contamination of H, deduplication effect, weighted Spearman; optional Markov/Monte Carlo
8	Part 2 report, README, `Makefile` reproducing every table
9	Buffer, review, presentation

- **Data.** One test leak (phpBB), English-speaking users, one training leak (RockYou). Both leaks reflect their own service and policy [1]. Robustness with a second test leak is optional.
- **HIBP circularity.** The leaks may be part of HIBP, as Golla and Dürmuth observe for their own data [1]. We measure the contamination of H in week 7 and read HIBP chiefly on R and L.
- **LLM population.** Results depend on the models, prompts, temperature and versions; the free, local models of at most 4 B parameters may differ from the commercial assistants studied in [4]; the optional manual samples check whether the duplicate rate carries over. A4 is a strong attacker that knows the model.
- **Implementation.** We use the zxcvbn Python port rather than the JavaScript reference of [1], [5]; the two agree on 3,012 passwords once the reference year is pinned, but only that sample was compared. Default configurations of the Linux tools are fixed in the repository.
- **Thresholds.** Budgets above 10^6 lie in the "don't care" region of [13]; results at 10^8-10^{10} test a meter's own scale rather than a modeled attack.

VII. Conclusion and Future Work

We reviewed eight related works, identified a gap (one-sided error rates of binary verdicts, Linux password-quality tools, LLM passwords with an LLM-aware attacker) and specified a protocol whose design is justified by the literature and whose feasibility is supported by measurements. The plan fits in nine weeks, uses only free resources, and the first end-to-end run already shows why both FPR and FNR are needed. The practical part will report the results, starting with the five meters on H, R and L at $\theta_{\text{on}} = 10^6$ and $\theta_{\text{off}} = 10^{10}$.

Future work. Three extensions follow from the limits above: stronger ground-truth attackers (Markov or PCFG models, and the Monte Carlo rank estimator of [7] once its variance at high ranks is controlled); a second test leak, to measure the dependence on the service; and LLM samples from larger or commercial models, together with targeted attackers that use profile data as in [14].

References

[1] M. Golla and M. Dürmuth, "On the accuracy of password strength meters," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Toronto, ON, Canada, 2018, pp. 1567-1582.

[2] D. Wang, X. Shan, Q. Dong, Y. Shen, and C. Jia, "No single silver bullet: Measuring the accuracy of password strength meters," in *Proc. 32nd USENIX Security Symposium*, Anaheim, CA, USA, 2023, pp. 947-964.

[3] D. Temoshok, J. Fenton, Y.-Y. Choong, N. Lefkovitz, A. Regenscheid, R. Galluzzo, and J. Richer, "Digital identity guidelines: Authentication and authenticator management," National Institute of Standards and Technology, NIST Special Publication 800-63B-4, Jul. 2025.

[4] Irregular, "Vibe password generation: Predictable by design," https://www.irregular.com/research/vibe-password-generation, Feb. 2026, accessed: Oct. 30, 2026.

[5] D. L. Wheeler, "zxcvbn: Low-budget password strength estimation," in *Proc. 25th USENIX Security Symposium*, Austin, TX, USA, 2016, pp. 157-173.

[6] B. Ur, S. M. Segreti, L. Bauer, N. Christin, L. F. Cranor, S. Komanduri, D. Kurilova, M. L. Mazurek, W. Melicher, and R. Shay, "Measuring real-world accuracies and biases in modeling password guessability," in *Proc. 24th USENIX Security Symposium*, Washington, DC, USA, 2015, pp. 463-481.

[7] M. Dell'Amico and M. Filippone, "Monte Carlo strength evaluation: Fast and reliable password checking," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Denver, CO, USA, 2015, pp. 158-169.

[8] M. Xu, W. Han, J. Yu, J. Liu, X. Zhang, Y. Lin, and J. S. Dong, "On the account security risks posed by password strength meters," arXiv:2505.08292 [cs.CR], 2025.

[9] Have I Been Pwned, "API v3: Pwned passwords," https://haveibeenpwned.com/API/V3, accessed: Oct. 30, 2026.

[10] libpwquality contributors, "libpwquality: password quality checking library," https://github.com/libpwquality/libpwquality, accessed: Oct. 30, 2026.

[11] Openwall, "passwdqc: password/passphrase strength checking and policy enforcement toolset," https://www.openwall.com/passwdqc/, accessed: Oct. 30, 2026.

[12] J. Tan, L. Bauer, N. Christin, and L. F. Cranor, "Practical recommendations for stronger, more usable passwords combining minimum-strength, minimum-length, and blocklist requirements," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Virtual Event, USA, 2020, pp. 1407-1426.

[13] D. Florêncio, C. Herley, and P. C. van Oorschot, "Pushing on string: The "don't care" region of password strength," *Communications of the ACM*, vol. 59, no. 11, pp. 66-74, 2016.

[14] M. Atzori, E. Calò, L. Caruccio, S. Cirillo, G. Polese, and G. Solimando, "Password strength analysis through social network data exposure: A combined approach relying on data reconstruction and generative models," arXiv:2511.16716, 2025.

[15] Ollama, "Ollama model library: Llama 3.2," https://ollama.com/library/llama3.2, accessed: Oct. 30, 2026.

[16] D. Miessler *et al.*, "SecLists," https://github.com/danielmiessler/SecLists, accessed: Oct. 30, 2026.

[17] hashcat contributors, "hashcat v6.2.6, advanced password recovery," https://hashcat.net/hashcat/, accessed: Oct. 30, 2026.

[18] Anthropic, "I have a paid Claude subscription (Pro, Max, Team or Enterprise plans), why is Claude API usage billed separately from my paid Claude plan?" https://support.claude.com/en/articles/9876003, accessed: Oct. 30, 2026.

[19] Google, "Gemini Developer API pricing," https://ai.google.dev/gemini-api/docs/pricing, accessed: Oct. 30, 2026.

[20] Google, "Gemini API terms of service," https://ai.google.dev/gemini-api/terms, accessed: Oct. 30, 2026.